ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Assignment I

Smart Energy Grid Analytics

PostgreSQL 17.11 with TimescaleDB 2.30.1 in Docker

Group members

Name	Student ID	Contribution
UWAYEZU Bienvenu	101362	Sections 1, 2, 3 and 9: ingestion pipeline, data capture and cleaning, chunk-interval comparison, final packaging.
UMUHOZA Fabiola	101361	Sections 4, 6 and 7.1-7.3: compression comparison, continuous aggregates, Raw and Aggregate dashboards.
NKUSI Paulin	101397	Sections 5, 8 and 7.4: utility analysis, anomalies, forecasting, dashboard performance timings.

Contents

1. Build the ingestion pipeline
2. Capture and prepare the historical dataset
3. Compare chunk intervals
4. Compare compression strategies
5. Investigate utility data
6. Continuous aggregates
7. Dashboards
8. Forecast the next day
9. Submission and reproducibility

Evidence checklist

List of figures

Figure 1. Docker Desktop containers: timescaledb (5433:5432) and the eclipse-mosquitto broker (1883) running, 14/10/2026 14:04.

Figure 2. docker ps: running containers, images and ports, 16/10/2026.

Figure 3. docker inspect mosquitto: bind-mounted config, data and log folders and start command, 16/10/2026.

Figure 4. mosquitto.conf: listener 1883, anonymous access disabled, password file, persistence enabled.

Figure 5. pipeline.py meters: meters=4000, 15/10/2026 12:25.

Figure 6. Database connection set-up, pipeline.py meters (meters=4000) and Test-NetConnection to port 1883 (TcpTestSucceeded: True), 16/10/2026 08:15.

Figure 7. Select-String search of publisher.py, subscriber.py and pipeline.py: broker credentials come from MQTT_USERNAME and MQTT_PASSWORD environment variables, 16/10/2026.

Figure 8. pipeline.py pilot: 2,000 generated, acknowledged, received and stored at QoS 1, retain false, 16/10/2026 08:23.

Figure 9. Full capture completion: consumed, stored and expected rows of 10,696,848, with CAT start and end times, 15/10/2026 12:31.

Figure 10. Data-quality diagnostics and the five meters with the most missing readings, 15/10/2026 12:36.

Figure 11. Cleanup run (clean.sql): 53,113 duplicate rows deleted, 10,643,735 rows remaining.

Figure 12. Workload query output: hourly average kW for 21 Sep 2026, 15/10/2026 12:37.

Figure 13. Workload query output: top network demand intervals and per-meter energy totals, 15/10/2026 12:38.

Figure 14. benchmark.py run for the one-day, three-hour and one-week tables, 15/10/2026 12:47.

Figure 15. explain_plans.py: Q5 plans saved for the three-hour and one-week tables, 15/10/2026 12:48.

Figure 16. explain_plans.py: Q5 plans saved for the time-only and meter-segmented compressed tables, 15/10/2026 12:52.

Figure 17. analyze_utility.py output: daily network energy by CAT day, 14/10/2026 13:52.

Figure 18. Complete analyze_utility.py output: weekly totals, peak intervals, group totals, completeness and the 21 Sep cutoff, 15/10/2026 12:55.

Figure 19. analyze_anomalies.py run: 40/40 events detected, 0 false flags, 15/10/2026 12:59.

Figure 20. pgAdmin: bgw_job table showing compression and continuous-aggregate refresh policy jobs.

Figure 21. refresh_aggregates.py: refresh time, row count and size for the three continuous aggregates, 15/10/2026 13:05.

Figure 22. freshness_test.py output, 15/10/2026 13:06.

Figure 23. Dashboard panel: hourly regional energy, 22-28 Sep 2026, meter 1000000000.

Figure 24. Dashboard panel: daily network energy.

Figure 25. Dashboard panel: seven-day network energy.

Figure 26. Dashboard panel: daily energy for meter 1000000000.

Figure 27. dashboard_compare.py: Raw versus Aggregate comparison for the five panels, 15/10/2026 13:08.

Figure 28. analyze_forecast.py run: splits, lag-source check and validation comparison, 14/10/2026 13:47.

Figure 29. Project folder listing and results files, 16/10/2026 08:11.

Assignment I: Smart Energy Grid Analytics

Database: PostgreSQL 17.11 with TimescaleDB 2.30.1 in Docker

1. Build the ingestion pipeline

1.1 Environment

The host record identifies an HP ENVY x360 2-in-1, 12th Gen Intel Core i7-1250U (10 cores, 12 logical processors), 16,077 MB RAM, Windows 11 Pro build 26200, and Python 3.14.4. PostgreSQL 17.11 and TimescaleDB 2.30.1 ran in Docker Desktop; the TimescaleDB container had no explicit CPU or memory limit. Recorded PostgreSQL settings are shared_buffers 1,947 MB, work_mem 5,192 kB, max_parallel_workers_per_gather 6, and jit off. Docker reported 12 CPUs and about 8.17 GB memory. The database session uses Africa/Kigali. The physical storage medium remains to be confirmed. The Windows clock was reported unsynchronized during the recorded work; synchronize it and re-record CAT evidence before submission.

The assignment specifies EMQX. The saved environment record says the pilot used Mosquitto 2.1.2 instead, and the Docker Desktop screenshot in Figure 1 shows the broker container running the eclipse-mosquitto image (port 1883), alongside the timescaledb container (host port 5433 mapped to 5432). Figure 2 lists the running containers with their images and ports, Figure 3 shows the mosquitto container's bind-mounted config, data and log folders and its start command, and Figure 4 shows its configuration: listener on port 1883, anonymous access disabled, a password file, and persistence enabled. This is a requirement deviation; repeat the pilot on EMQX if available, otherwise state the deviation accurately.

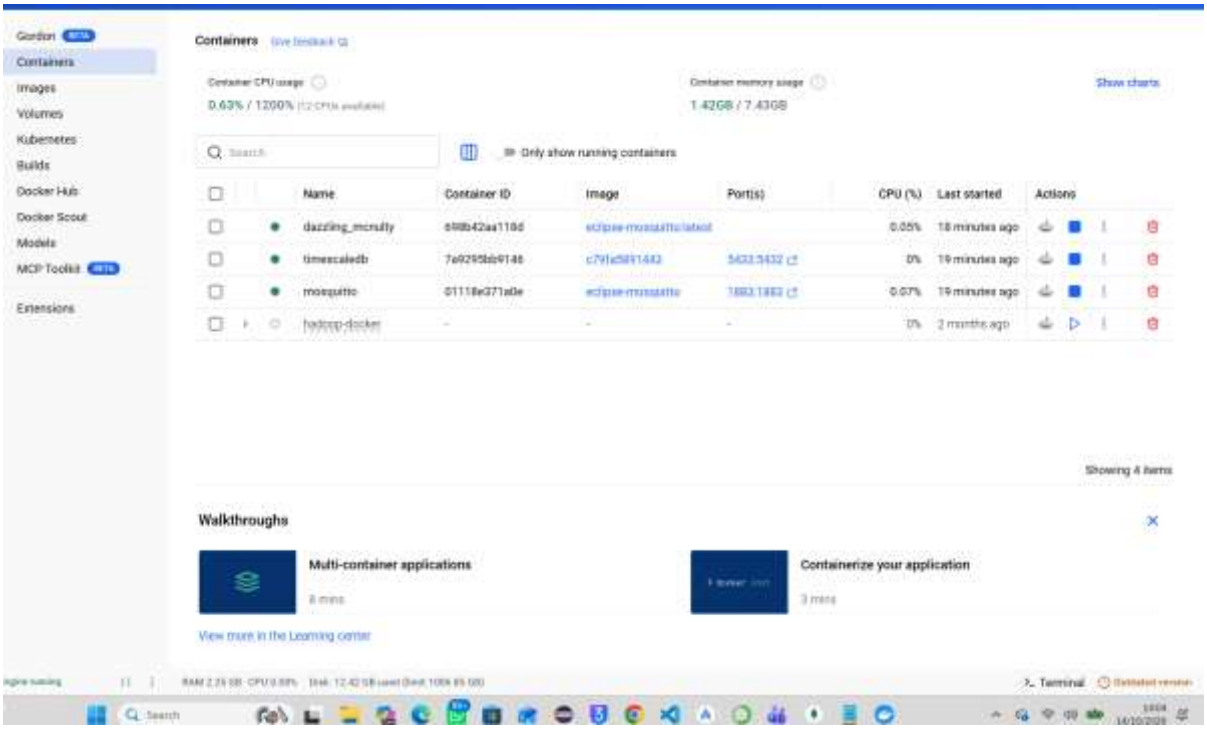


Figure 1. *Docker Desktop containers: timescaledb (5433:5432) and the eclipse-mosquitto broker (1883) running, 14/10/2026 14:04.*



Figure 2. *docker ps: running containers, images and ports, 16/10/2026.*

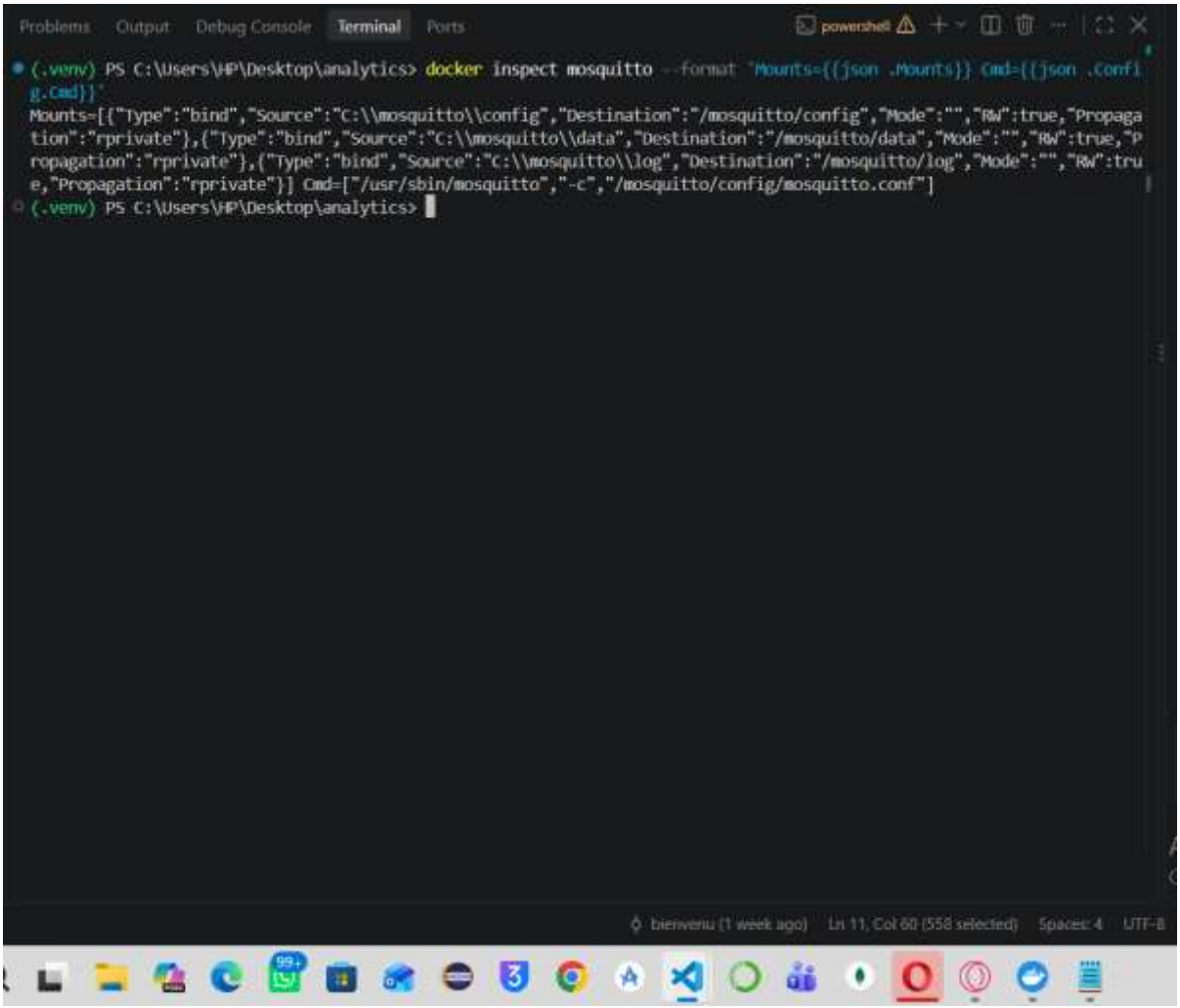


Figure 3. *docker inspect mosquitto: bind-mounted config, data and log folders and start command, 16/10/2026.*

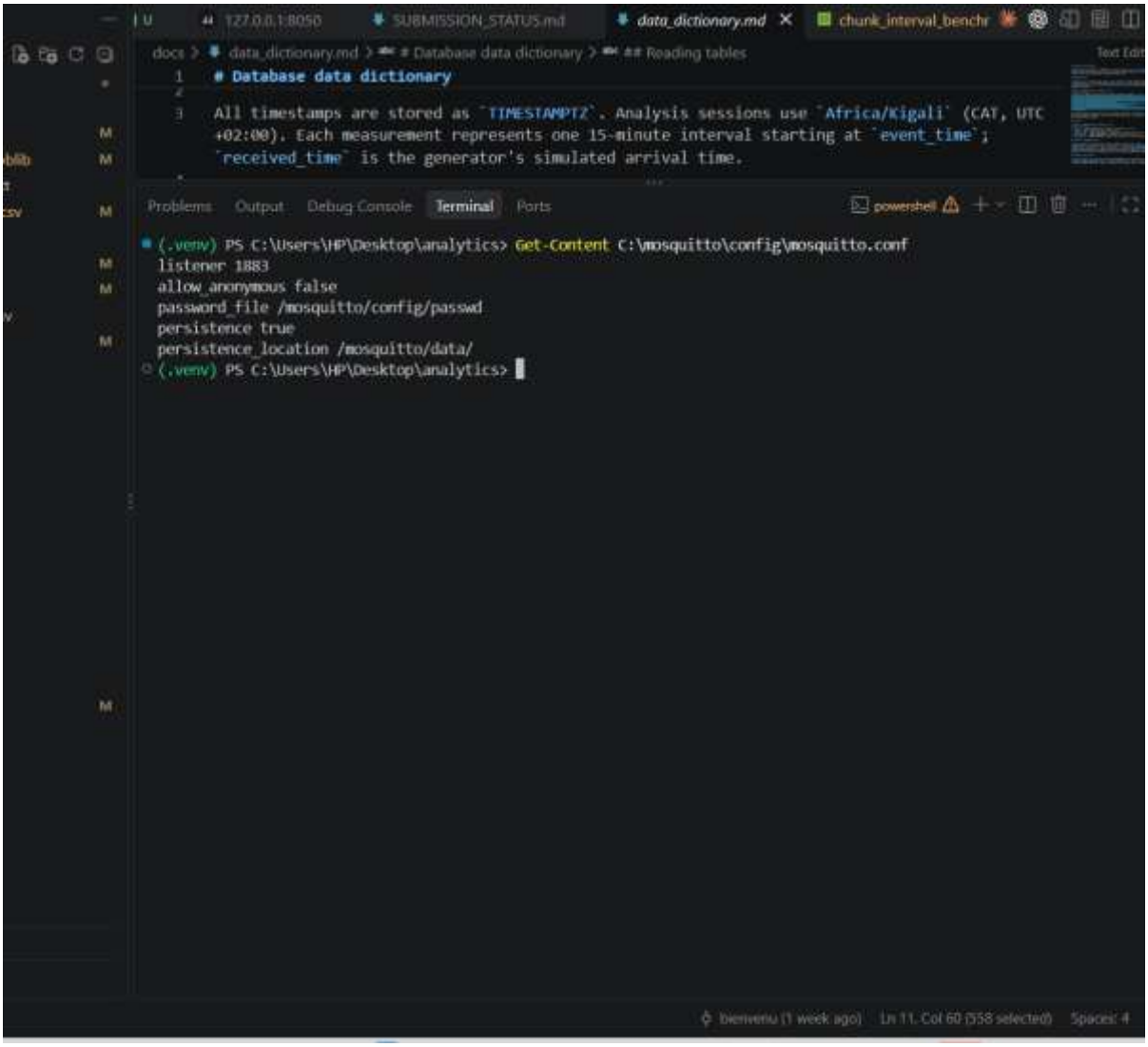


Figure 4. *mosquitto.conf: listener 1883, anonymous access disabled, password file, persistence enabled.*

Evidence to insert: broker version output (the screenshots show the image name eclipse-mosquitto but not the version); enabled TimescaleDB extension; session time zone; host/system details and clock.

1.2 Database structures

The schema defines energy_live, energy_readings, and meters. energy_readings is a one-day Timescale hypertable with time and meter/time indexes. The metadata loader reads the simulator's 4,000 meter records; the pipeline.py meters command in Figure 5 reports meters=4000. The data dictionary documents columns, types, meanings, and units in docs/data_dictionary.md.



Figure 5. *pipeline.py meters: meters=4000, 15/10/2026 12:25.*

Evidence to insert: table definitions and types, and the one-day chunk interval.

1.3 Pilot stream

The pilot run on 16 Oct 2026 (08:23:54.101 to 08:23:57.446 CAT, 3.345 seconds) reported 2,000 generated, 2,000 acknowledged, 2,000 received, and 2,000 stored, at QoS 1 with retain disabled (Figure 8). Before it, Test-NetConnection confirmed that TCP port 1883 on localhost was reachable (Figure 6). The publisher, subscriber and pipeline scripts read the broker credentials from the MQTT_USERNAME and MQTT_PASSWORD environment variables instead of storing them in the code (Figure 7). The standalone publisher and subscriber counts also show 2,000 at their stages.

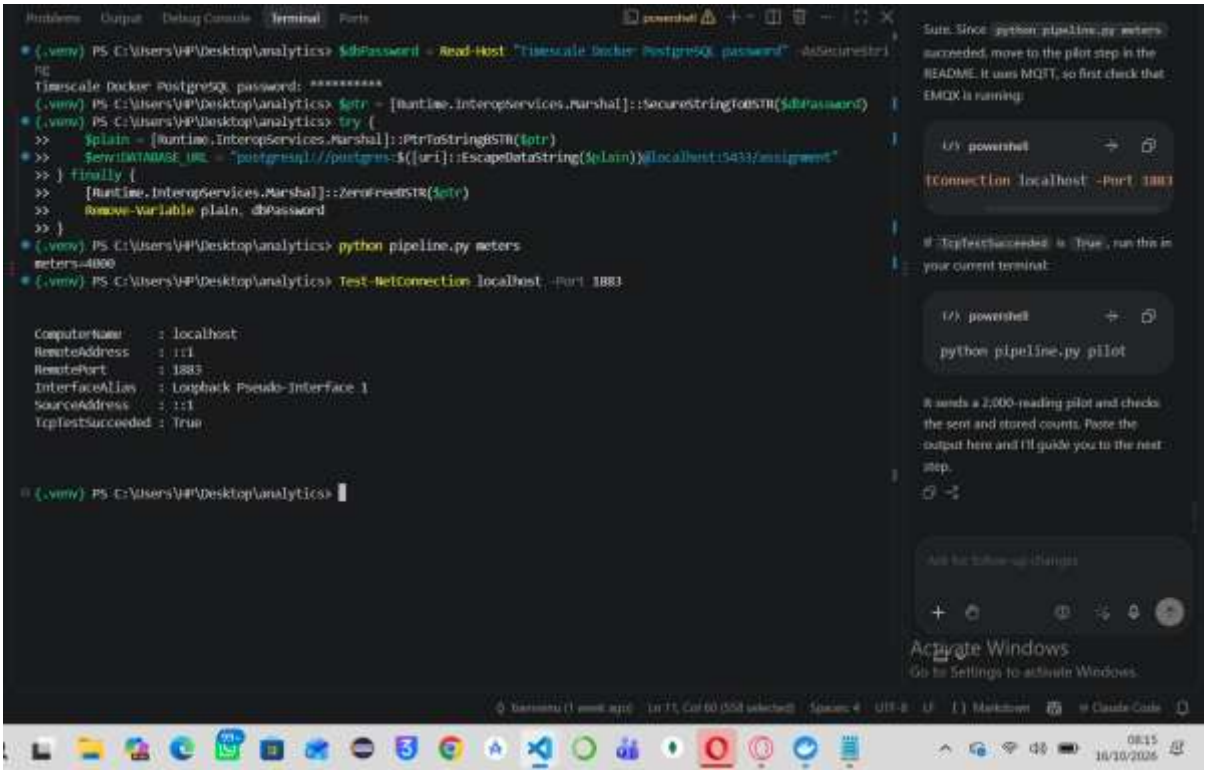


Figure 6. *Database connection set-up, pipeline.py meters (meters=4000) and Test-NetConnection to port 1883 (TcpTestSucceeded: True), 16/10/2026 08:15.*

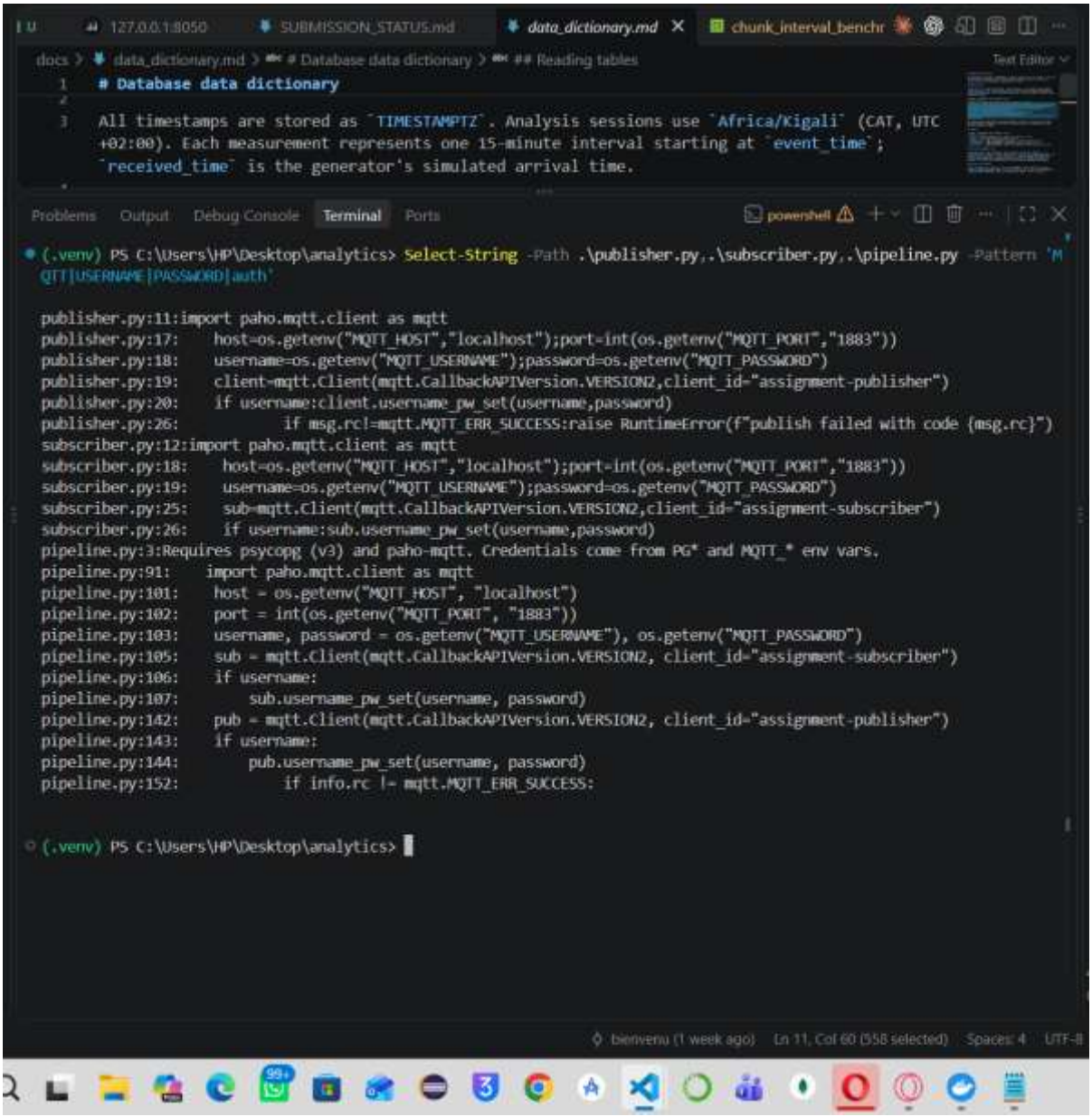


Figure 7. *Select-String search of publisher.py, subscriber.py and pipeline.py: broker credentials come from MQTT_USERNAME and MQTT_PASSWORD environment variables, 16/10/2026.*

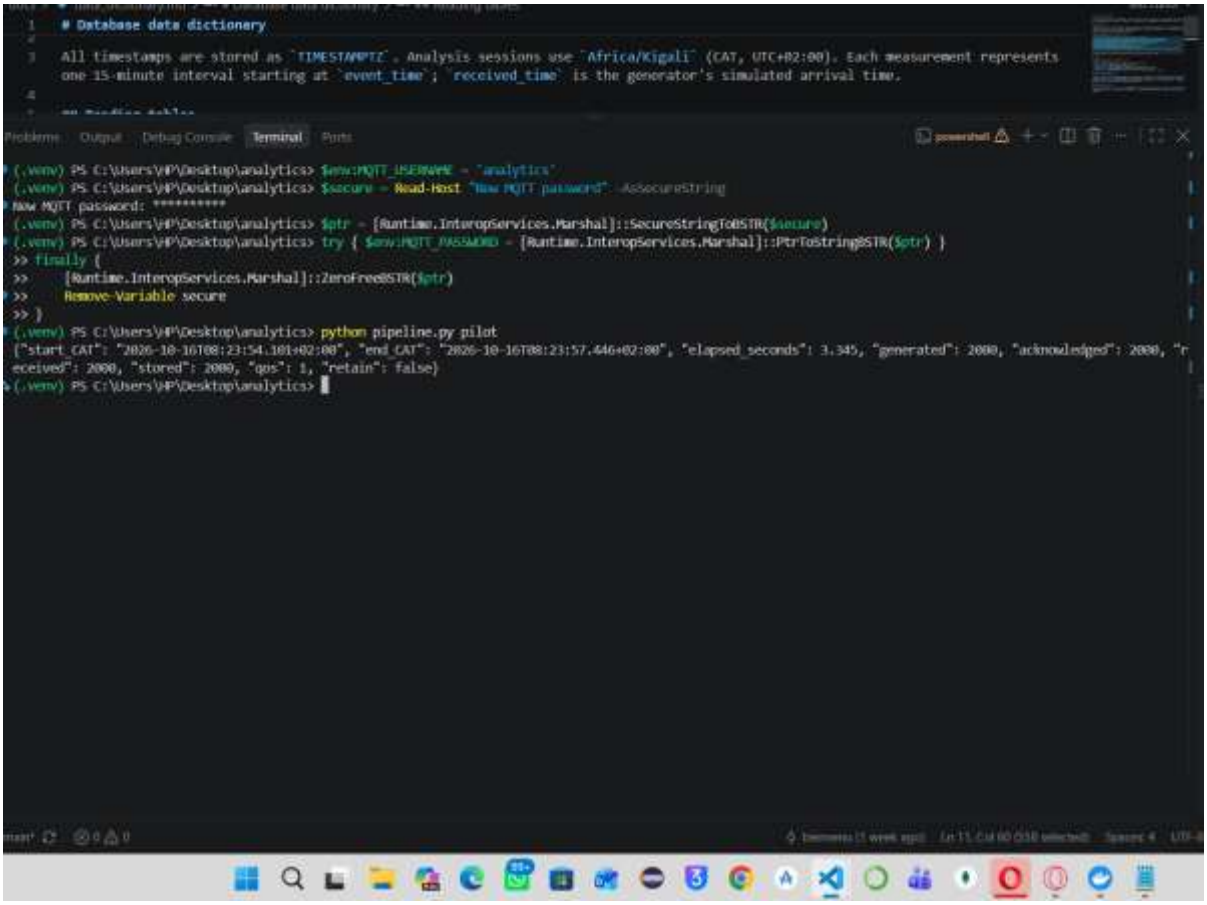


Figure 8. *pipeline.py pilot: 2,000 generated, acknowledged, received and stored at QoS 1, retain false, 16/10/2026 08:23.*

Evidence to insert: publisher completion, subscriber completion, and stored row/sample query.

2. Capture and prepare the historical dataset

2.1 Full capture

The direct PostgreSQL COPY capture on DESKTOP-CK0HHMD consumed and stored 10,696,848 rows, matching the simulator's expected count (Figure 9). It ran from 12:26:04.337 to 12:28:33.453 CAT on 15 Oct 2026; total time was 149.02 seconds, of which 148.843 seconds were write time. The cleaned dataset contains 10,643,735 rows. The pilot remains separate in energy_live.

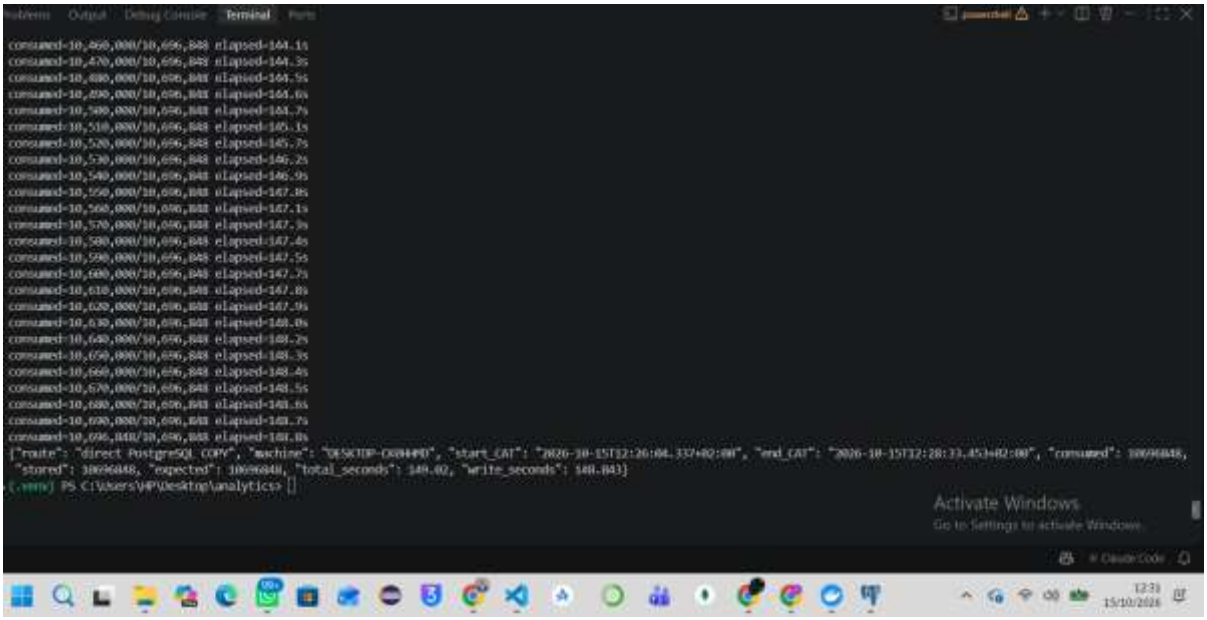


Figure 9. *Full capture completion: consumed, stored and expected rows of 10,696,848, with CAT start and end times, 15/10/2026 12:31.*

Evidence to insert: date range, distinct meters, and table/index/total storage sizes.

2.2 Data quality

Before cleaning, the source contained 10,696,848 emitted rows, 10,643,735 unique meter/time pairs, 53,113 extra copies, 213,090 delayed unique readings, 108,265 missing planned pairs, and no conflicting duplicate groups. Missing rate was 1.006929%, duplicate rate 0.499007%, delayed rate 2.002023%, and completeness 98.993071%. The five highest missing-meter counts are saved in results/top_five_missing_meters.csv. Figure 10 shows the diagnostic query output and the five meters with the most missing readings.

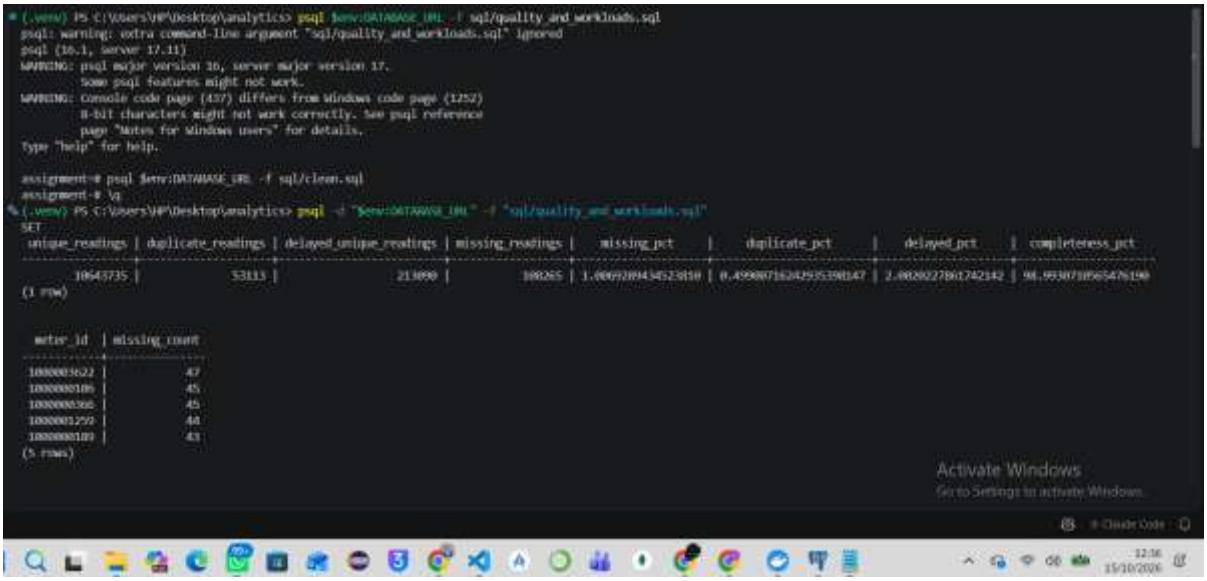


Figure 10. *Data-quality diagnostics and the five meters with the most missing readings, 15/10/2026 12:36.*

2.3 Cleaning

The cleanup retained one row per meter/time pair after confirming duplicate payloads did not conflict. Rows fell from 10,696,848 to 10,643,735; summed energy changed from 3,872,863.893 kWh to 3,853,704.347 kWh because repeated copies were removed. Gaps were preserved rather than filled with zero, since a missing measurement is unknown consumption, not evidence of no consumption. Figure 11 shows the cleanup run (53,113 rows deleted) and the check that rows after cleaning equal unique meter/time pairs (10,643,735), with a total of 3,853,704.347 kWh.

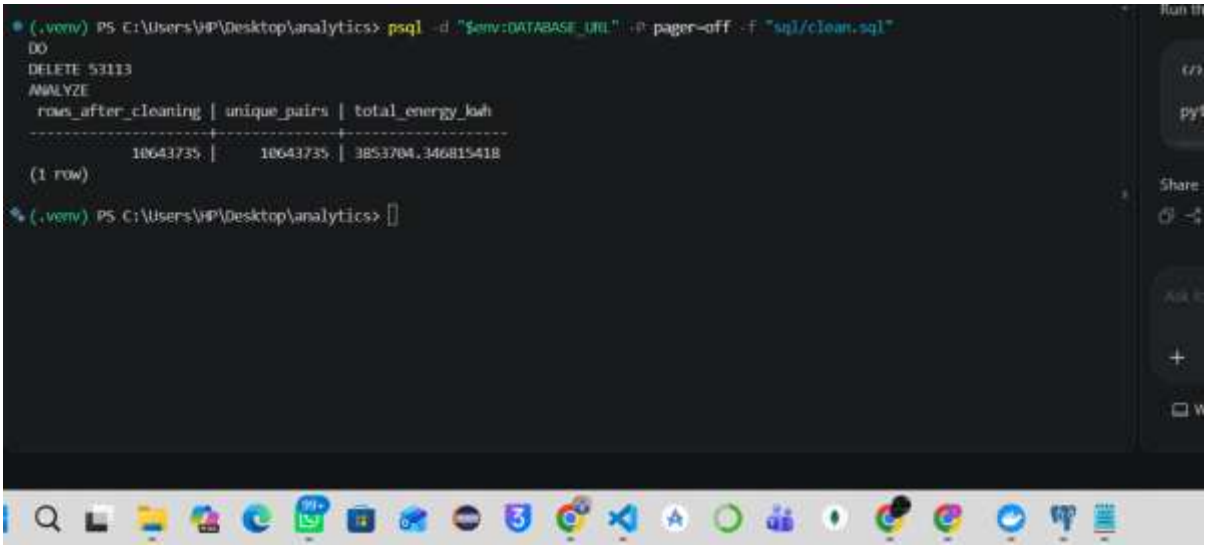


Figure 11. *Cleanup run (clean.sql): 53,113 duplicate rows deleted, 10,643,735 rows remaining.*

Evidence to insert: the before-cleanup energy total (3,872,863.893 kWh).

3. Compare chunk intervals

3.1 Configurations and data checks

The project defines one-day energy_readings, three-hour energy_readings_3h, and one-week energy_readings_week hypertables, with matching event-time and meter/event-time indexes. The saved benchmark workflow includes correctness checks and Q5 plans for all three configurations.

Evidence to insert: chunk interval/count, index definitions, date range and matching row/meter/sum checks for all three tables.

3.2 Workload timings

Saved runs use psycopg 3 / Python on DESKTOP-CK0HHMD and contain warm-up plus five measured runs per query/table. Times below are medians in milliseconds; full min/max and timestamps are in results/chunk_interval_benchmark_summary.csv and results/chunk_interval_benchmark_runs.csv.

Query	1 day	3 hours	1 week	Fastest
Q1	33.62	32.08	58.35	3 hours
Q2	103.93	100.41	197.67	3 hours
Q3	429.34	1,214.69	648.19	1 day
Q4	3.33	5.92	4.24	1 day
Q5	288.03	493.93	456.17	1 day

Figures 12 and 13 show outputs of the workload queries in sql/quality_and_workloads.sql, Figure 14 shows the benchmark.py run for the three tables, and Figure 15 shows the Q5 plan files being saved for the three-hour and one-week tables.

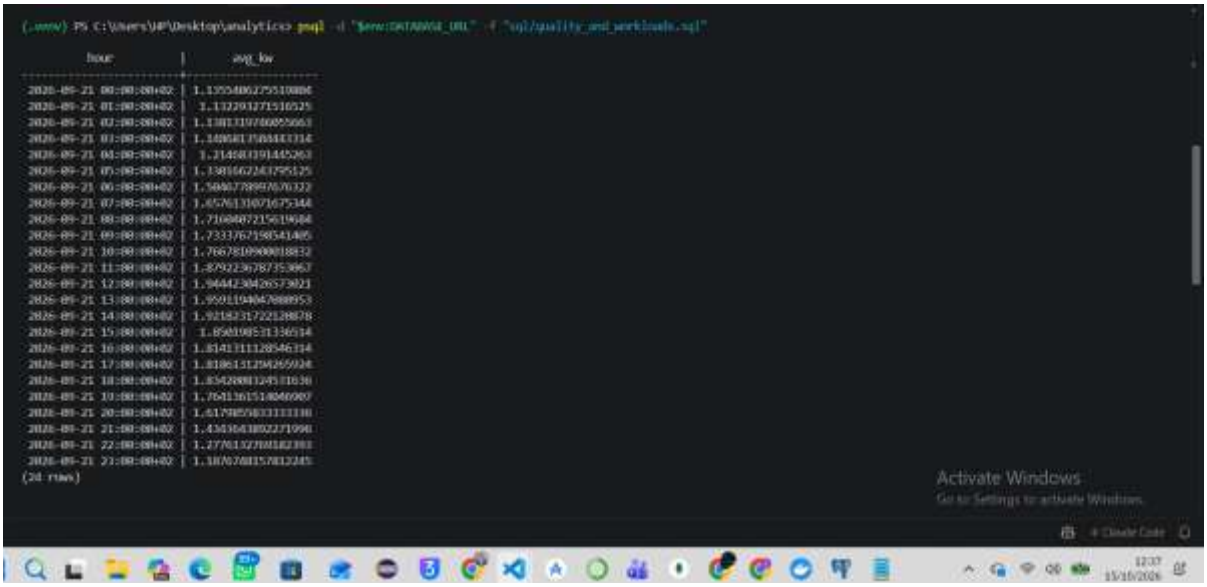


Figure 12. *Workload query output: hourly average kW for 21 Sep 2026, 15/10/2026 12:37.*

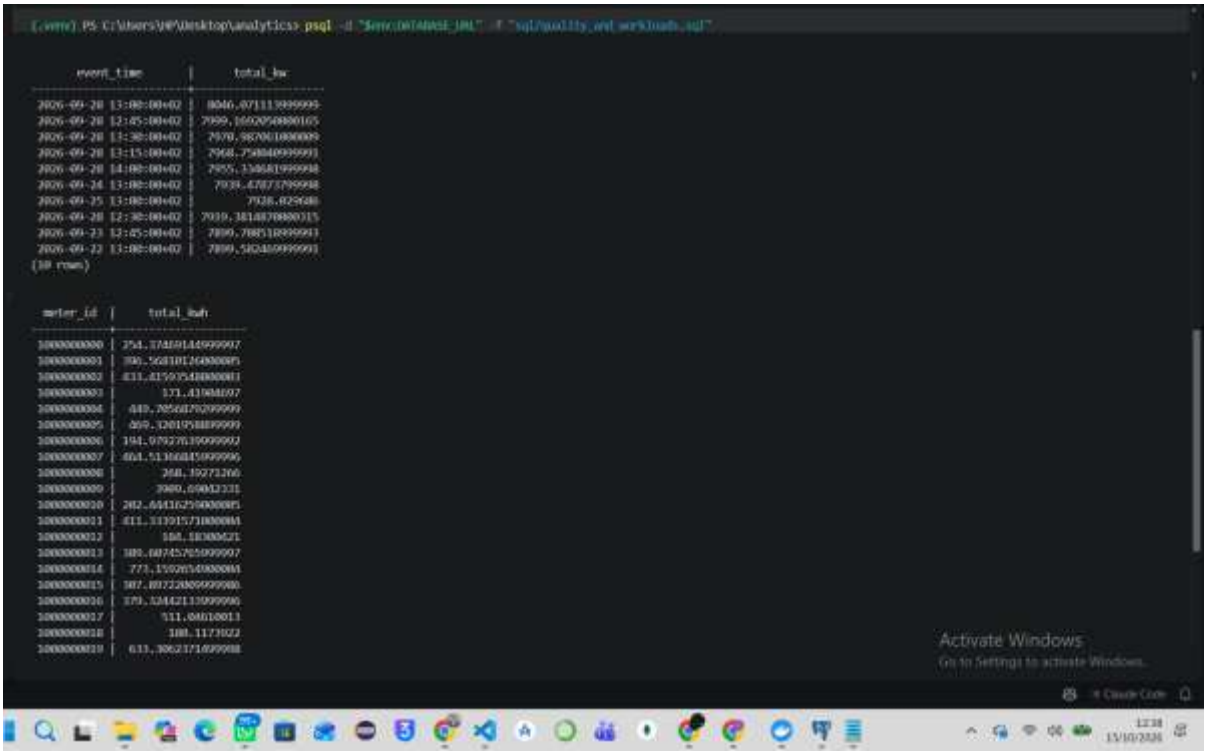


Figure 13. *Workload query output: top network demand intervals and per-meter energy totals, 15/10/2026 12:38.*

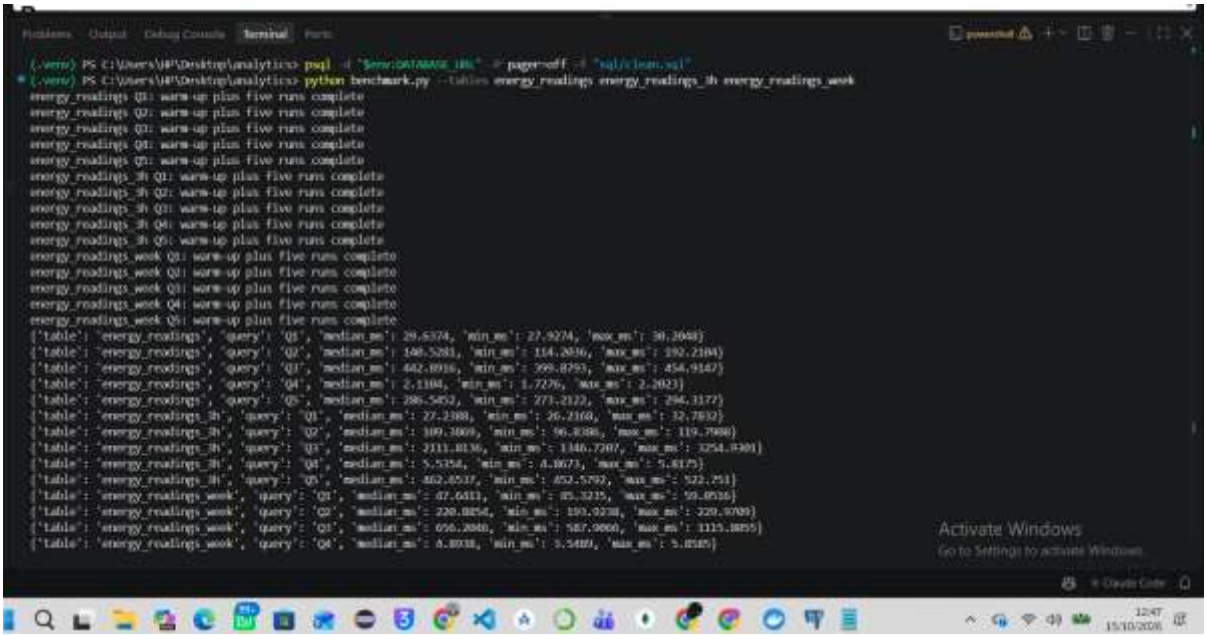


Figure 14. *benchmark.py run for the one-day, three-hour and one-week tables, 15/10/2026 12:47.*



Figure 15. *explain_plans.py: Q5 plans saved for the three-hour and one-week tables, 15/10/2026 12:48.*

Evidence to insert: the contents of one saved timing/buffer plan per table (Figure 15 shows only that the files were saved). Label all results warm-cache.

Check: the medians in Figure 14 differ from the table above (for example one-day Q1 29.64 ms against 33.62 ms, and three-hour Q3 2,111.82 ms against 1,214.69 ms), so the table appears to come from an earlier run. Update the table and Section 3.3 from the latest results/ files, or recapture, so the report and screenshots agree. The Q5 one-week row is cut off in the screenshot.

3.3 Interpretation

The three-hour chunks were slightly faster for the two time-filtered workloads Q1 and Q2, by about 4.6% and 3.4% versus one day. One-day chunks were fastest for Q3, Q4, and Q5. The one-week layout was slower on all five measured queries. This is consistent with the workload mix: smaller time chunks can reduce time-range scanning, while many smaller chunks add planning/aggregation work to broad queries. These are interpretations of this machine's measured results, not universal guarantees.

4. Compare compression strategies

4.1 Layout and loading

Two one-day compressed hypertables were built from the cleaned data: event-time order without segmentation, and event-time order segmented by meter_id. Saved logs show both layouts have the same 29 time ranges compressed and contain 10,643,735 rows. Policies were paused for the measurements.

Evidence to insert: compression settings, policy configuration and paused state, chunk ranges, and data equivalence checks.

4.2 Storage and compression cost

Layout	Total size	Ratio vs. 1-day baseline	Saved	Compressed rows/chunks	Compression time
Uncompressed baseline	1,503.85 MB	1.00x	-	-	-
Time-only	460.28 MB	3.27x	69.39%	100%; 29/29	12.84 s
Meter-segmented	526.26 MB	2.86x	65.01%	100%; 29/29	43.61 s

The baseline and compressed data/index sizes are recorded in results/compression_storage_summary.csv.

Evidence to insert: storage statistics and chunk compression timing summary.

4.3 Query comparison

Medians in milliseconds, against the same one-day uncompressed baseline:

Query	Baseline	Time-only	Meter-segmented
Q1	29.15	24.97	219.56
Q2	107.10	74.72	62.21
Q3	421.18	174.06	126.01
Q4	2.38	43.71	2.76
Q5	284.89	182.76	207.92

Time-only compression improved Q1, Q2, Q3, and Q5 by 14.4%, 30.2%, 58.7%, and 35.8%, but made selected-meter Q4 slower. Meter segmentation improved Q2, Q3, and Q5 by 41.9%, 70.1%, and 27.0%; Q4 was 16.1% slower than baseline and Q1 was much slower. Full comparison and actual Q5 plans are saved under results/.

Figure 16 shows the Q5 plan files being saved for both compressed tables.



Figure 16. *explain_plans.py: Q5 plans saved for the time-only and meter-segmented compressed tables, 15/10/2026 12:52.*

Evidence to insert: comparison results, the contents of the plan files, and policy enabled status after measurements.

4.4 Recommendation

For historical reporting, choose one-day time-only compression: it achieved 3.27x total-size reduction and faster medians for four of five workloads, with 12.84 seconds total measured compression cost. Keep an uncompressed one-day table for frequent selected-meter reads. Meter segmentation is an alternative if that query dominates; it approached baseline Q4 latency but used more storage and took longer to compress.

5. Investigate utility data

5.1 Demand patterns

Weekly network energy increased over the four periods: 934,199.061 kWh (1 Sep), 953,714.972 kWh (8 Sep), 973,230.026 kWh (15 Sep), and 992,560.288 kWh (22 Sep). The top network demand interval was 28 Sep 2026 at 13:00 CAT, 7,993.691 kW; the next were 12:45 at 7,943.947 kW and 13:15 at 7,943.636 kW. kW describes interval-average demand; kWh sums consumed energy. Figure 17 shows the daily energy output of analyze_utility.py, and Figure 18 shows the complete analyze_utility.py output, including the weekly totals and the three peak intervals.

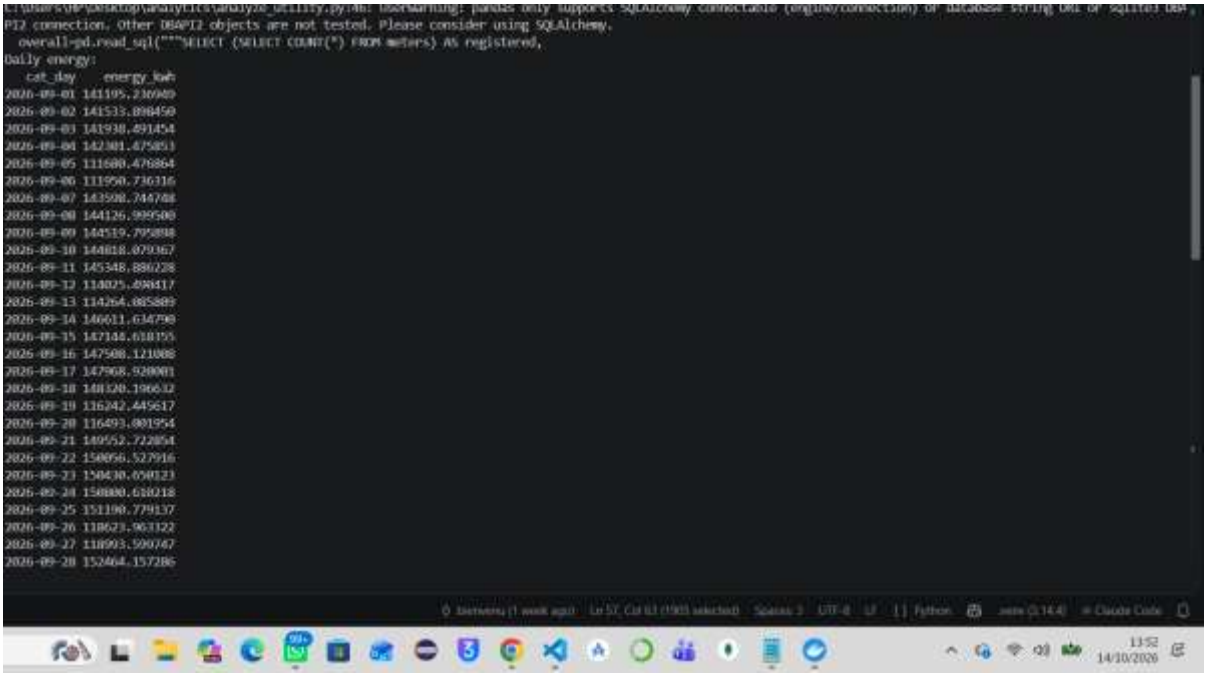


Figure 17. *analyze_utility.py output: daily network energy by CAT day, 14/10/2026 13:52.*

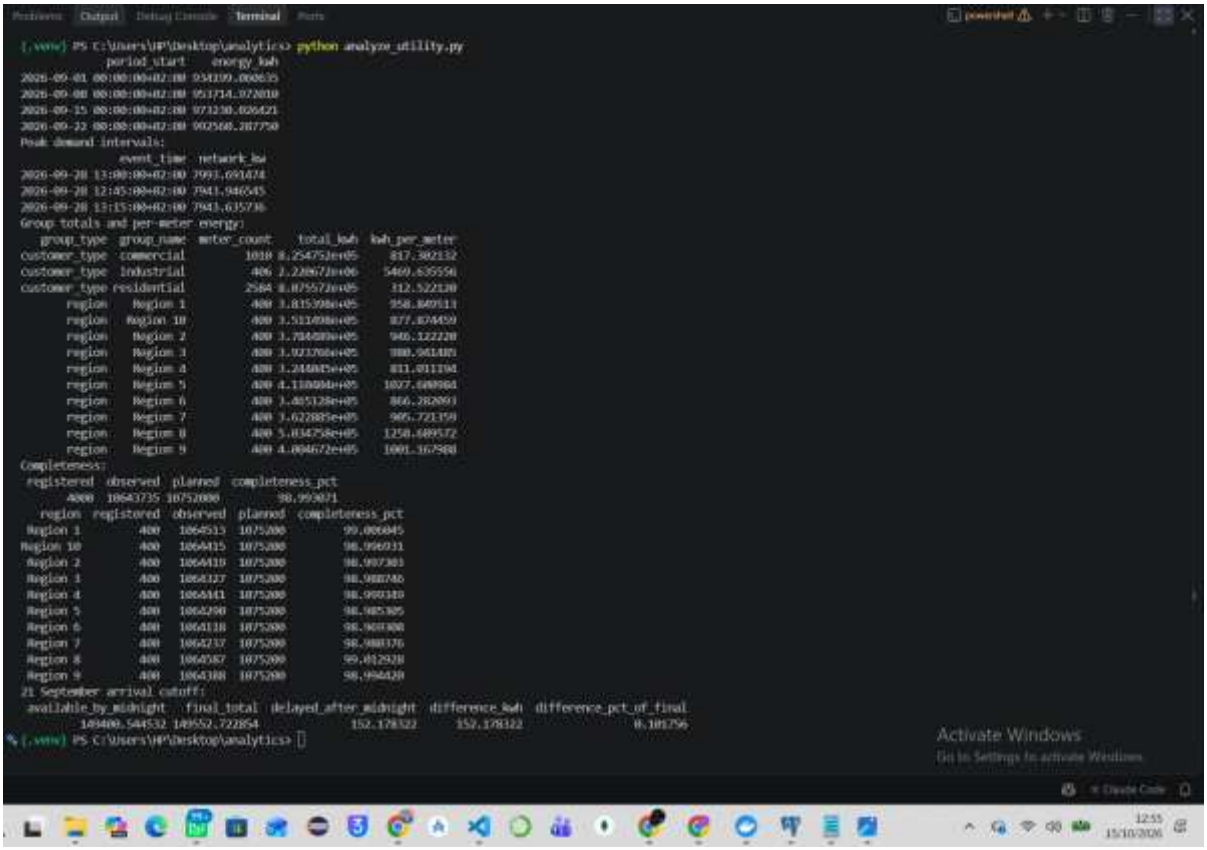


Figure 18. *Complete analyze_utility.py output: weekly totals, peak intervals, group totals, completeness and the 21 Sep cutoff, 15/10/2026 12:55.*

Evidence to insert: daily/weekly charts.

5.2 Regions and customer types

Each region has 400 registered meters. Region 8 was highest at 1,258.690 kWh per meter and Region 4 lowest at 811.011 kWh per meter. Customer-type totals were: industrial 2,220,672.036 kWh across 406 meters (5,469.636 kWh/meter); commercial 825,475.153 kWh across 1,010 meters (817.302 kWh/meter); residential 807,557.158 kWh across 2,584 meters (312.522 kWh/meter). Differences persist after normalization, so the group totals are not explained solely by group size. The group totals and per-meter values are shown in Figure 18.

5.3 Reporting quality

Overall completeness was 98.993071%. Regional completeness ranged from 98.969308% (Region 6) to 99.012928% (Region 8). For 21 Sep, 149,400.545 kWh had arrived by midnight on 22 Sep, versus a finalized 149,552.723 kWh. The late-arrival difference was 152.178 kWh, or 0.101756% of the final total. Missing and late readings can make early reports

incomplete. Overall and regional completeness and the 21 September arrival cutoff comparison are shown in Figure 18.

5.4 Anomalies

The rule compares each reading with the same meter's median for the same CAT hour and weekday. It flags a ratio at least 2.3 or at most 0.10. Event labels were used only after detection for evaluation. The saved findings report 40/40 injected intervals detected, 0 false flags among 10,643,261 readings outside event intervals, and 0 of 4,000 meters affected by false flags. Figure 19 shows the detector run.

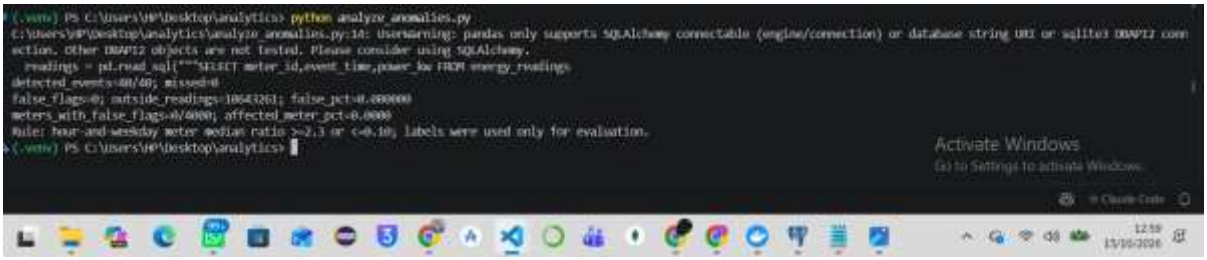


Figure 19. *analyze_anomalies.py run: 40/40 events detected, 0 false flags, 15/10/2026 12:59.*

Evidence to insert: flagged rows (sample) if required.

5.5 Management recommendation

Use the one-day time-only compressed layout for historical reports and retain an uncompressed one-day table for selective recent reads. The time-only layout reduced total size by 69.39% and improved four of five query medians, with the selected-meter query as the tradeoff. For next-day planning, use the selected RF_300 model as decision support. On the latest saved test metrics, its MAE/RMSE were 5.122/7.126 kWh versus 42.708/79.403 kWh for the previous-day baseline. Display freshness and missing-data limitations because finalized historical totals can exceed early live totals. Use regional and customer-type demand patterns to focus operational load management and capacity review. Reassess with additional seasons and live-arrival replay before operational deployment.

6. Continuous aggregates

6.1 Definitions and refresh

Three materialized-only aggregates are defined: 15-minute region, CAT hour region, and CAT day meter. Saved initial full-refresh times and materialization sizes are:

View	Refresh	Rows	Materialized total
energy_15m_region	15.60 s	26,880	4,349,952 bytes
energy_hour_region	7.09 s	6,720	1,294,336 bytes
energy_day_meter	12.06 s	112,000	13,164,544 bytes

Refresh timings were recorded before policies were installed. The SQL scripts define the policy schedule and lookback/end offsets. Figure 20 shows the TimescaleDB background-job catalog (_timescaledb_catalog.bgw_job) viewed in pgAdmin: three Refresh Continuous Aggregate Policy jobs (1007, 1008 and 1009) with a 15-minute schedule interval, and two Columnstore (compression) policy jobs (1002 and 1003) with a 12-hour schedule interval, alongside the built-in telemetry and job-history retention jobs.

Figure 21 shows the refresh_aggregates.py run.

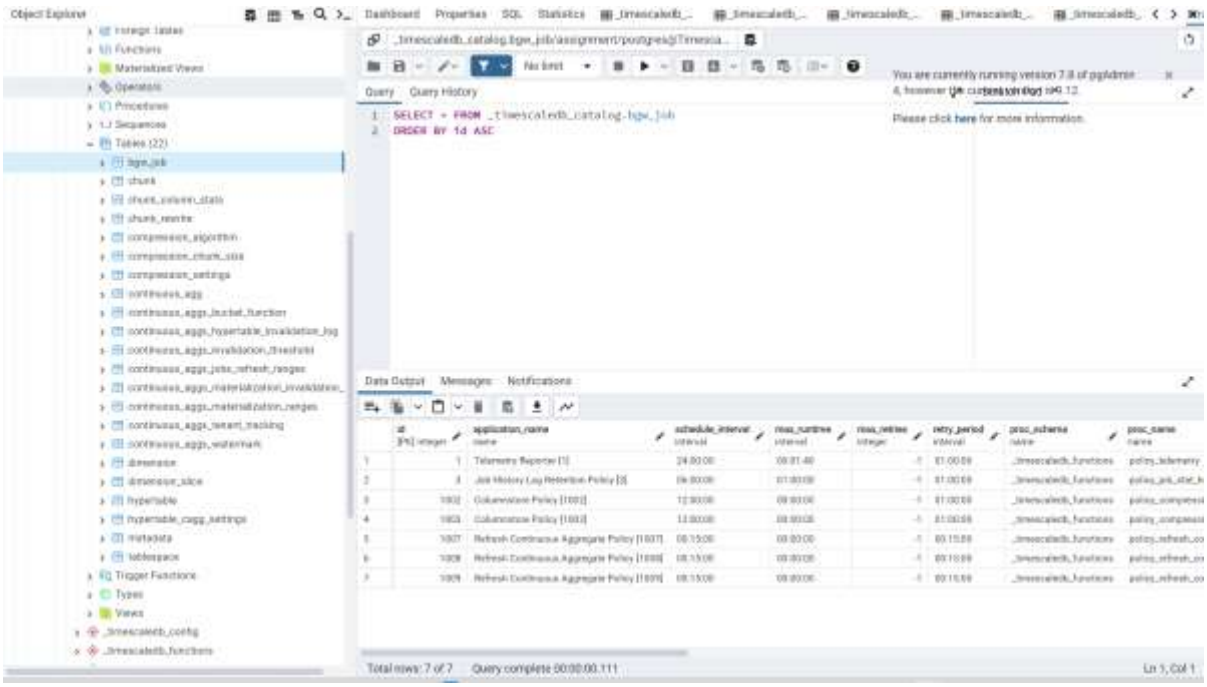


Figure 20. *pgAdmin: bgw_job table showing compression and continuous-aggregate refresh policy jobs.*

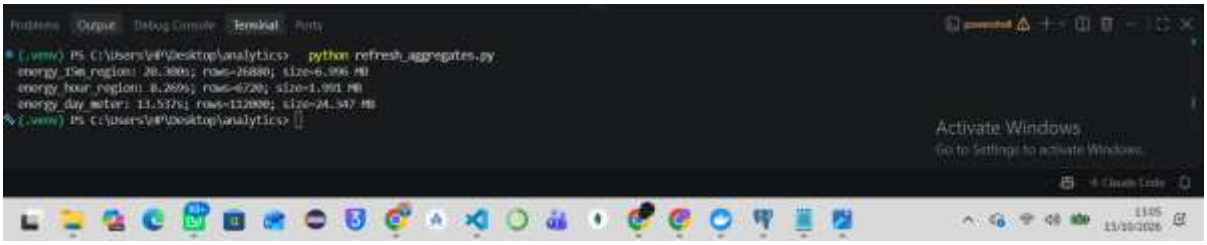


Figure 21. *refresh_aggregates.py: refresh time, row count and size for the three continuous aggregates, 15/10/2026 13:05.*

Evidence to insert: view definitions, populated examples, and the policy offsets and job enabled/scheduled status (not visible in Figure 20).

Check: Figure 21 reports 20.300 s, 8.269 s and 13.537 s and sizes of 6.996, 1.991 and 24.347 MB, which differ from the refresh times and sizes in the table above (row counts match). Update the table from the latest run or explain the difference.

6.2 Freshness and delayed arrivals

At 11:03:16 CAT on 2 Oct, H was 10:00 and the older test hour was 10:00 on 30 Sep. The first older-hour insert aggregated to 1 kWh/one row. After adding 0.5 kWh to that older bucket and 2 kWh to H, with real-time aggregation off, stored values remained 1 kWh for the older bucket and no H bucket. With real-time aggregation on, H showed 2 kWh/one row while the older bucket remained stale. After manually refreshing both hours, the older bucket showed 1.5 kWh/two rows and H showed 2 kWh/one row.

Figure 22 shows the freshness_test.py output.



Figure 22. *freshness_test.py output, 15/10/2026 13:06.*

Evidence to insert: definitions, inserts, real-time mode, and raw/aggregate query output for each stage.

Check: Figure 22 comes from a run on 15 Oct 2026 at 13:06:25 CAT (completed hour H = 12:00, older hour 13 Oct 12:00), but the description above is from the 2 Oct run (H = 10:00, older hour 30 Sep 10:00). Make the description and screenshot refer to the same run. The first part of the output line is cut off.

7. Dashboards

7.1 Raw and 7.2 Aggregate dashboards

The Dash app provides Raw and Aggregate modes with date/meter filters and five panels: hourly regional energy, daily network energy, weekly network energy, selected-meter daily energy, and regional completeness. Raw mode queries the base readings; Aggregate mode uses the continuous aggregates. The app source and panel queries are in dashboard.py. Figures 23 to 26 show four of the panels for whole CAT days 22 to 28 Sep 2026 and meter 1000000000.

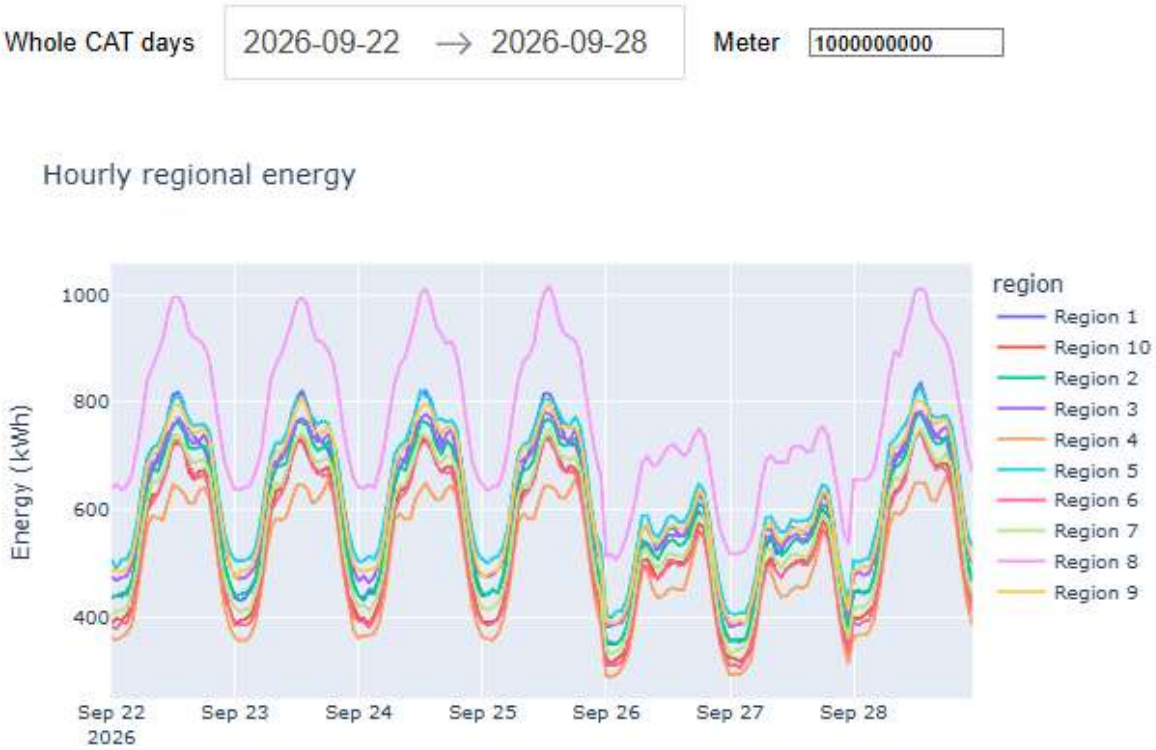


Figure 23. *Dashboard panel: hourly regional energy, 22-28 Sep 2026, meter 1000000000.*

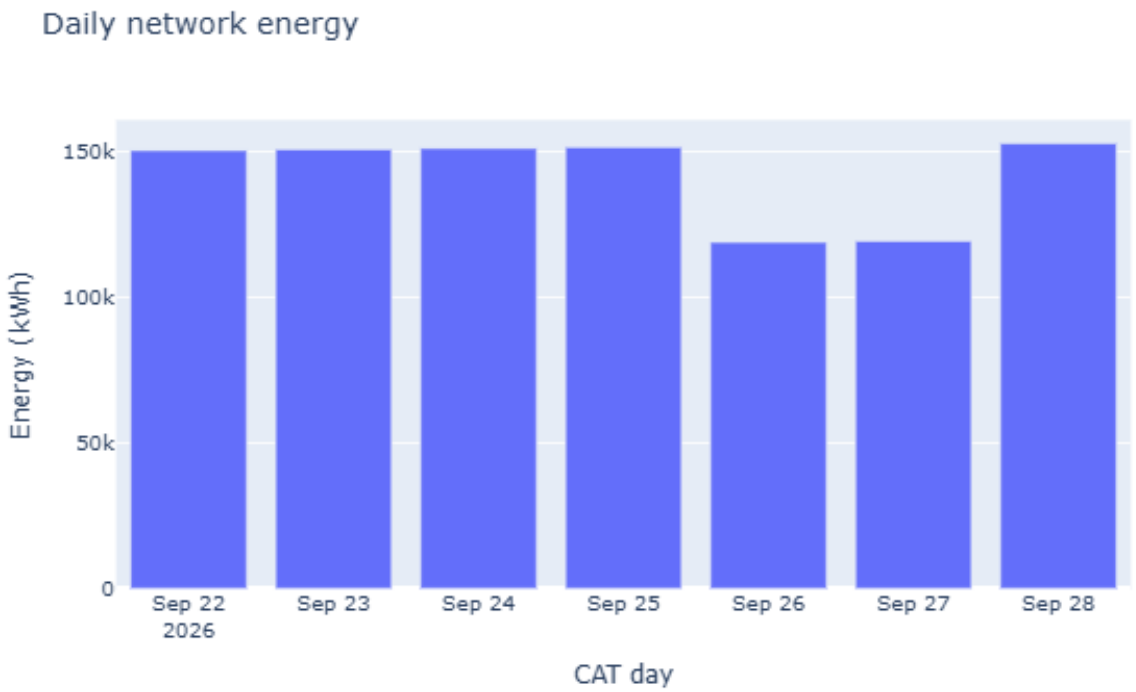


Figure 24. *Dashboard panel: daily network energy.*

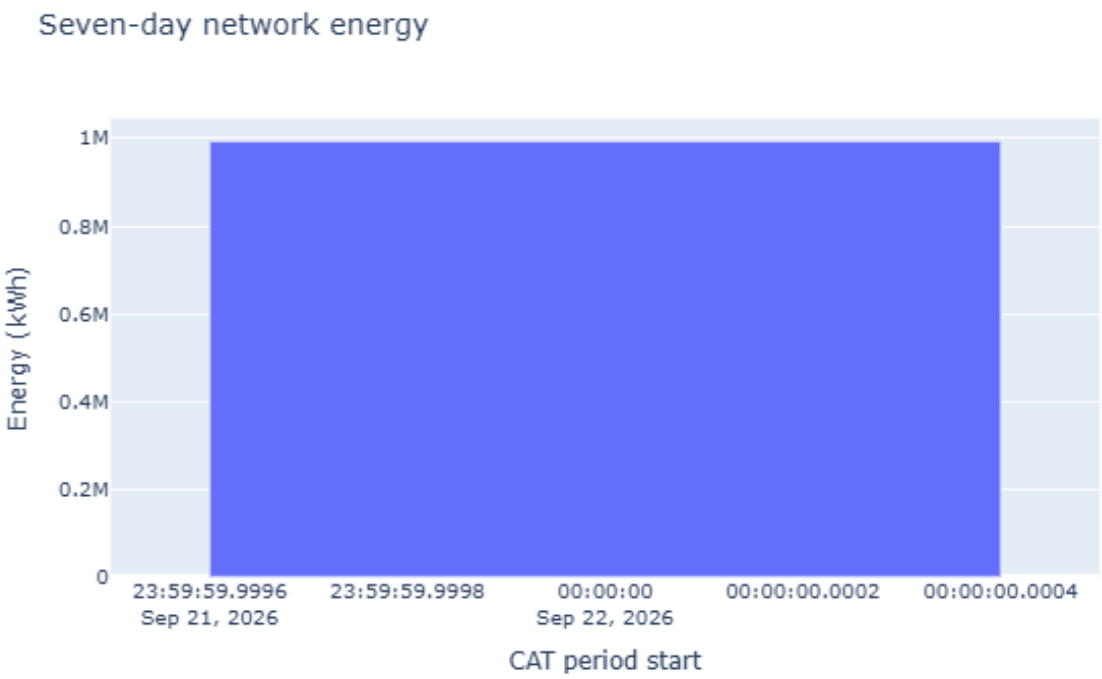


Figure 25. *Dashboard panel: seven-day network energy.*



Figure 26. *Dashboard panel: daily energy for meter 1000000000.*

Evidence to insert: the regional completeness panel, the mode selector showing which mode (Raw or Aggregate) each capture was taken in, and the panel query definitions.

7.3 Numerical agreement

For 22-29 Sep, the saved comparison found matching keys and counts across all five panels. It compared 1,680 hourly-region rows, 7 daily rows, 1 weekly row, 7 selected-meter daily rows, and 10 regional-completeness rows. Maximum energy differences were below 0.000001 kWh and there were no reading-count mismatches; completeness difference was zero.

Figure 27 shows the dashboard_compare.py output: all five panels have zero reading-count mismatches, and the largest energy difference is 6.17e-09 kWh.



Figure 27. *dashboard_compare.py: Raw versus Aggregate comparison for the five panels, 15/10/2026 13:08.*

Evidence to insert: paired filtered dashboards.

7.4 Performance tradeoff - outstanding

The project record says browser refresh performance was not measured. Assignment evidence requires one warm-up and ten measured refreshes per mode, individual panel query durations, full-dashboard render duration, and median/min/max. Do not substitute SQL benchmark timings for browser-render timings. Capture these on the same machine/browser with result caching disabled and assignment jobs paused. Then compare the timing cost with aggregate storage and refresh costs above.

8. Forecast the next day

8.1 Data, splits, and features

The script creates hourly regional targets from cleaned readings, preserving absent target hours as missing. It splits targets chronologically: 1-14 Sep train, 15-21 Sep validation, and 22-28 Sep test. The script reports 1,680 retained training rows after excluding 1,680 of 3,360 pre-lookback training rows; validation and test each contain 1,680 rows. Features are region, hour, weekday, day of year, 24-hour lag, 168-hour lag, and a prior-week same-hour mean. Imputation is fitted inside each training pipeline. Lag-source checks verify every lag is before that day's 00:00 forecast origin. Figure 28 shows the analyze_forecast.py run, including the split counts, the passed lag-source check, the earliest usable feature time (8 Sep 2026 00:00 CAT) and the validation comparison.



Figure 28. *analyze_forecast.py run: splits, lag-source check and validation comparison, 14/10/2026 13:47.*

Evidence to insert: feature catalog/examples.

8.2 Baseline and 8.3 Model

The previous-day matching-hour baseline generated 1,680 test predictions (1,680 non-null in Figure 28). Two configurations were trained on the same 1,680 rows: RF_300 and HGB_depth3. RF_300 had the lower validation MAE (4.678 kWh; RMSE 6.103 kWh) and was selected, against 5.549 kWh and 7.771 kWh for HGB_depth3. The selected fitted pipeline is saved at results/selected_forecast_model.joblib.

Evidence to insert: sample predictions; model settings.

8.4 Test usefulness

Latest saved test metrics across 1,680 region-hours:

Method	MAE	RMSE
Previous-day baseline	42.708 kWh	79.403 kWh
RF_300	5.122 kWh	7.126 kWh

Region-level metrics, peak-hour errors, predictions, and plot are saved under results/. These results indicate a substantial improvement on the fixed historical test split. Peak-hour error and missing-reading limitations still matter for operational planning; this is an offline forecast of finalized historical consumption.

Evidence to insert: metric outputs, regional peak errors, and forecast plot.

9. Submission and reproducibility

The repository contains the unchanged simulator, source scripts, SQL, README, environment notes, and saved measurements. Before submission, add the final group names, IDs, and contribution statement; capture every subsection's required screenshot with synchronized CAT clock and taskbar; complete dashboard timings; verify EMQX requirement or document the deviation; record physical storage type; and create/check the final report PDF, GitHub repository URL, and ZIP against the same final file set. Keep credentials out of both packages. Figure 29 shows the current project folder listing (scripts, sql, docs, results, README.md and requirements.txt).

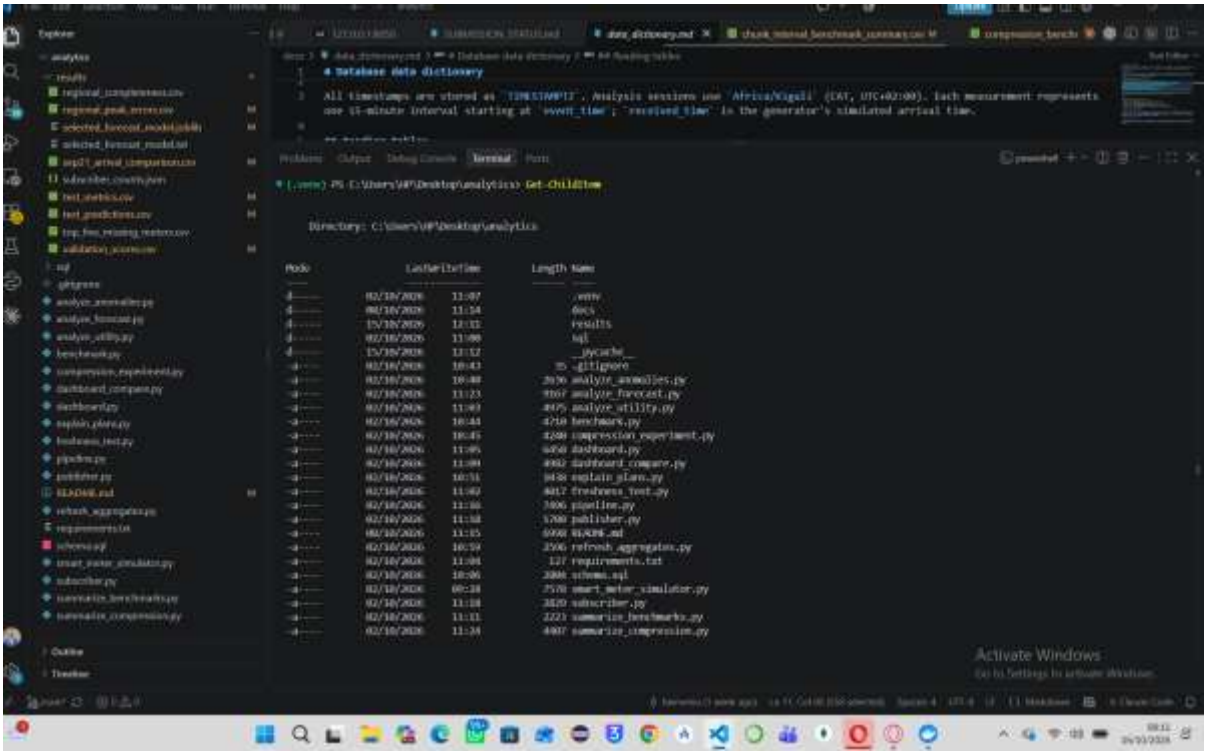


Figure 29. *Project folder listing and results files, 16/10/2026 08:11.*

Evidence to insert: final package inventory (ZIP contents) and repository URL.

Evidence checklist

Each bracketed evidence note above must be replaced with the required real screenshot(s) beside its subsection. The existing charts and CSVs are analysis outputs, not substitutes for screenshots of executed commands, database checks, or dashboard sessions. Confirm host clock synchronization before recording new timings.